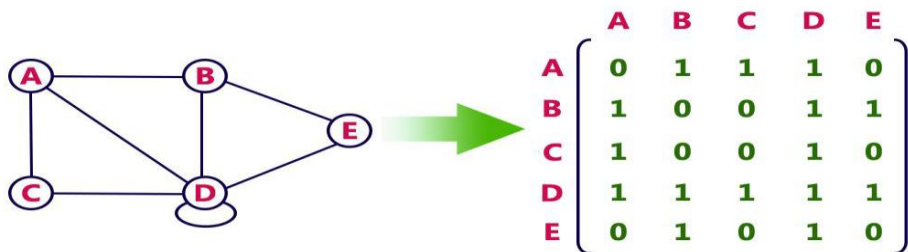


## • Graphs

- Graph is a data structure that consists of following two components:
  - A finite set of vertices (nodes).
  - A finite set of edge(ordered pair of the form  $(u, v)$  )
- A graph  $G = (V, E)$ , where **V** is a set of vertices and **E** is a set of edges.
- Representations of graph.
  - Adjacency Matrix
  - Adjacency List

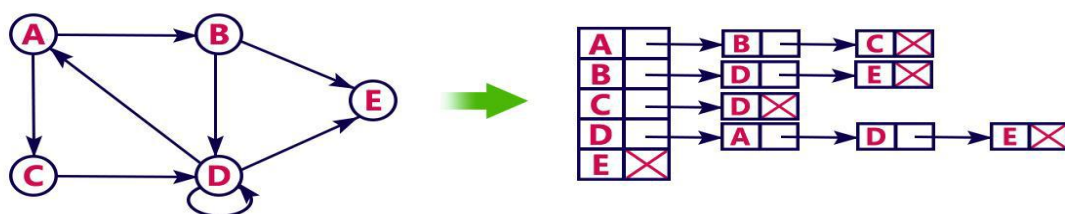
### ○ Adjacency Matrix:

- Adjacency Matrix is a 2D array(say  $adj[i][j]$ ) of size  $|V| \times |V|$  where  $|V|$  is the number of vertices in a graph.
- If  $adj[i][j] = 1$ , then there is an edge from vertex  $i$  to vertex  $j$ .
- Adjacency matrix for undirected graph is always symmetric.
- Adjacency Matrix is also used to represent weighted graphs.
  - If  $adj[i][j] = w$ , then there is an edge from vertex  $i$  to vertex  $j$  with weight  $w$ .



### ○ Adjacency List:

- An array of linked lists is used.
- Size of the array is equal to number of vertices.
- An entry  $array[i]$  represents the linked list of vertices adjacent to the  $i^{th}$  vertex.
- This representation can also be used to represent a weighted graph. The weights of edges can be stored in nodes of linked lists.



### ○ Types of graph:

- Undirected Graph: A graph with only undirected edges.
- Directed Graph: A graph with only directed edges.
- Directed Acyclic Graphs(DAG): A directed graph with no cycles.
- Cyclic Graph: A directed graph with at least one cycle.
- Weighted Graph: It is a graph in which each edge is given a numerical weight.
- Disconnected Graphs: An undirected graph that is not connected.

- **Graph Traversal Algorithms:**

- Graph traversal algorithms visit the vertices of a graph, according to some strategy. Different graph traversal algorithms are:

- Breadth First Search(BFS)
- Depth First Search(DFS)

- **Breadth First Search(BFS)**

- **Algorithm BFS(G, u)**

1. Set all nodes are unvisited
2. Mark the starting vertex  $u$  as visited and put it into an empty Queue  $Q$
3. While  $Q$  is not empty
  - 3.1 Dequeue  $v$  from  $Q$
  - 3.2 While  $v$  has an unvisited neighbor  $w$ 
    - 3.2.1 Mark  $w$  as visited
    - 3.2.2 Enqueue  $w$  into  $Q$
4. If there is any unvisited node  $x$ 
  - 4.1 Visit  $x$  and Insert it into  $Q$ . Goto step 3

- **Complexity**

- If the graph is represented as an adjacency list
  - Each vertex is enqueued and dequeued atmost once. Each queue operation take  $O(1)$  time. So the time devoted to the queue operation is  **$O(V)$** .
  - The adjacency list of each vertex is scanned only when the vertex is dequeued. Each adjacency list is scanned atmost once. Sum of the lengths of all adjacency list is  $|E|$ . Total time spend in scanning adjacency list is  **$O(E)$** .
  - Time complexity of BFS =  $O(V) + O(E) = \mathbf{O(V+ E)}$ .
  - **In a dense graph:**
    - $E=O(V^2)$
    - Time complexity=  $O(V) + O(V^2) = \mathbf{O(V^2)}$
- If the graph is represented as an adjacency matrix
  - There are  $V^2$  entries in the adjacency matrix. Each entry is checked once.
  - Time complexity of BFS =  **$O(V^2)$**

- **Applications of BFS**

- Finding shortest path between 2 nodes  $u$  and  $v$ , with path length measured by number of edges
- Testing graph for bipartiteness
- Minimum spanning tree for unweighted graph
- Finding nodes in any connected component of a graph
- Serialization/deserialization of a binary tree
- Finding nodes in any connected component of a graph

- **Depth First Search(DFS)**

**Algorithm DFS(G, u)**

1. Mark vertex u as visited
2. For each adjacent vertex v of u
  - 2.1 if v is not visited
    - 2.1.1 DFS(G, v)

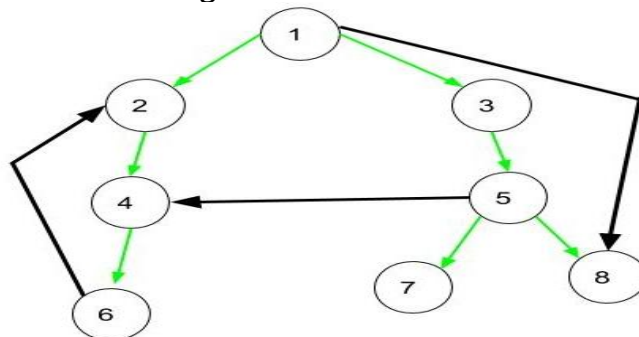
**Algorithm main(G,u)**

1. Set all nodes are unvisited.
2. DFS(G, u)
3. For any node x which is not yet visited
  - 3.1 DFS(G, x)

- **Complexity**

- If the graph is represented as an adjacency list
  - Each vertex is visited atmost once. So the time devoted is  $O(V)$
  - Each adjacency list is scanned atmost once. So the time devoted is  $O(E)$
  - Time complexity of DFS =  $O(V + E)$ .
- If the graph is represented as an adjacency matrix
  - There are  $V^2$  entries in the adjacency matrix. Each entry is checked once.
  - Time complexity of DFS =  $O(V^2)$

- **Classification of Edges**



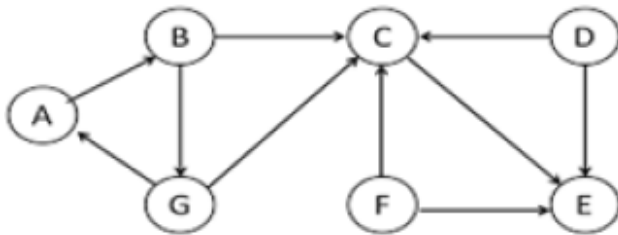
- The DFS traversal of the above graph is 1 2 4 6 3 5 7 8
- **Tree Edge:** It is a edge in tree obtained after applying DFS on the graph
  - Eg: (1,2), (2,4), (4,6), (1,3), (3,5), (5,7) and (5,8)
- **Forward Edge:**
  - It is an edge (u, v) such that v is descendant but not part of the DFS tree
  - Eg: (1,8)
- **Backward Edge**
  - It is an edge (u, v) such that v is ancestor of edge u but not part of DFS tree
  - Eg: (6,2)
- **Cross Edge**
  - It is a edge which connects two node such that they do not have any ancestor and a descendant relationship between them.
  - Eg: (5,4)

- **Applications of DFS**

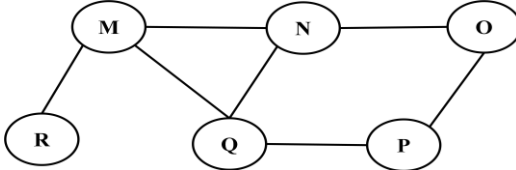
- Finding connected components in a graph
- Topological sorting in a DAG
- Scheduling problems
- Cycle detection in graphs
- Finding 2-(edge or vertex)-connected components
- Finding 3-(edge or vertex)-connected components
- Finding the bridges of a graph
- Finding strongly connected components
- Solving puzzles with only one solution, such as mazes
- Finding biconnectivity in graphs

- **University Questions**

1. Give Breadth First Search algorithm for graph traversal. Perform its complexity analysis
2. Write a short note on graph traversals. Perform BFS traversal on the below graph starting from node A. If multiple node choices may be available for next travel, choose the next node in alphabetical order

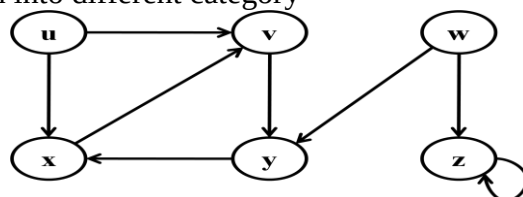


3. Write the BFS traversal of given graph. Consider M as start vertex

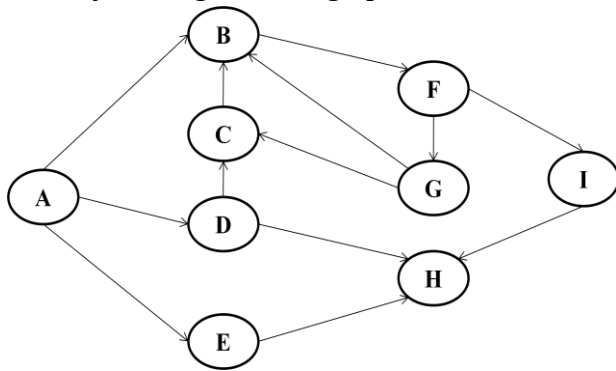


**Answer:** BFS Traversal is M, N, Q, R, O, P

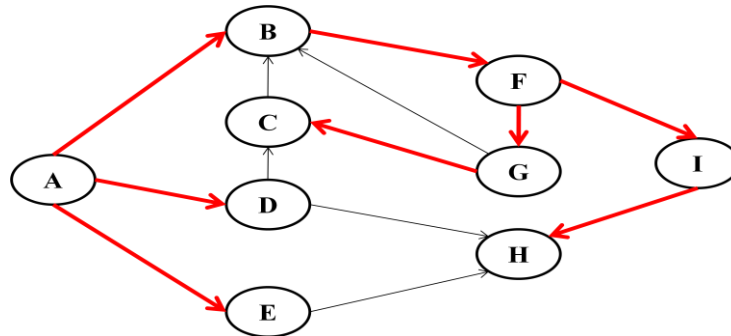
4. Give Depth First Search algorithm for graph traversal. Perform its time complexity analysis
5. Write DFS algorithm and analyse its time complexity. Illustrate the classification of edges in DFS traversal.
6. What are different classification of edges that can be encountered during DFS operation and how it is classified? Explain with example
7. Write down DFS algorithm and analyse the time complexity. What are classification of edges that can be encountered during DFS operation and how it is classified?
8. Perform DFS traversal on the following graph starting from node A. When multiple nodes are available for next traversal choose nodes in alphabetical order. Classify the edges of the graph into different category



9. Perform DFS traversal on the bellow graph starting from node A. Where multiple node choices may be available for next travel, choose the next node in alphabetical order. Classify the edges of the graph into different categories



**Answer:**



DFS Traversal: A B F G C I H D E

Tree Edge : (A,B), (B,F), (F,G), (G,C), (C,I), (I,H), (A,D), (A,E)

Forward Edge : --

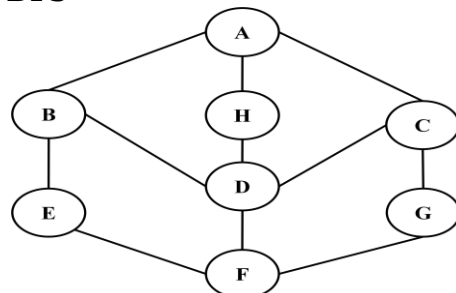
Backward Edge : (G,B), (C,B)

Cross Edge : (D,C), (D,H), (E,H)

10. Apply these algorithms on the following graph. Let A be the source vertex. Analyse complexity of each algorithm

i. **BFS**

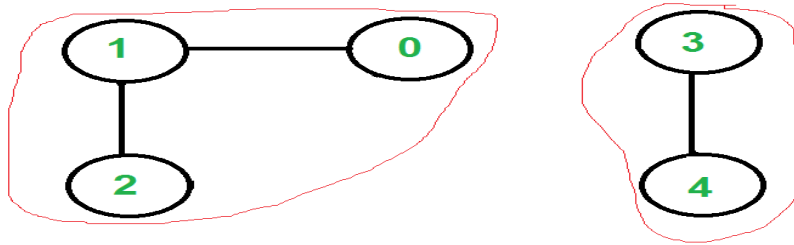
ii. **DFS**



11. Write the applications of BFS and DFS

- **Connected Components:**

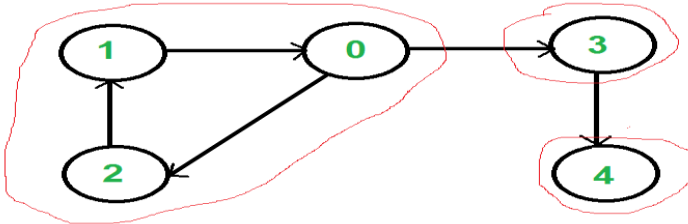
- Connected component of a graph  $G$  is a connected subgraph of  $G$  of maximum size
- A graph may have more than one connected components.



There are two connected components in above undirected graph  
 0 1 2  
 3 4

- **Strongly Connected Components(SCC)**

- Strong Connectivity applies only to directed graphs.
- A strongly connected component of a directed graph is a complement such that all the vertices in that component is reachable from every other vertex in that component.



Here we have 3 SCCs. {0,1,2}, {3}, {4}

- **Kosaraju's Algorithm**

- It is a 2 Pass algorithm. Steps 1-4 are Pass1. Steps 5-7 are Pass2.
- 1. Set all vertices of graph  $G$  are unvisited.
- 2. Create an empty stack  $S$ .
- 3. Do DFS traversal on unvisited vertices and set it as visited. If a vertex has no unvisited neighbor, push it in to the stack.
- 4. Perform the above step until all vertices are visited
- 5. Reverse the graph  $G$ .
- 6. Set all nodes are unvisited.
- 7. While  $S$  is not Empty
  - 7.1 POP one vertex  $v'$
  - 7.2 If  $v'$  is not visited
    - 7.2.1 Set  $v'$  as visited
    - 7.2.2 Call DFS( $v'$ ). It will print strongly connected component of  $v'$ .

- **Time Complexity**

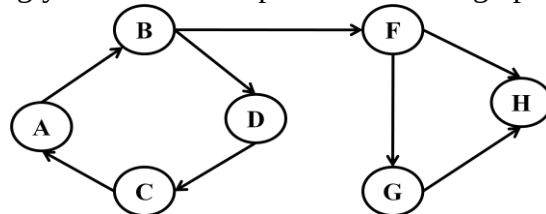
- First pass we did DFS. So the time complexity =  $O(V+E)$
- Reversal of a graph will take  $O(V+E)$  time
- Pass 2 will take another  $O(V+E)$  time
- Total time complexity =  $O(V+E)$

○ **Applications**

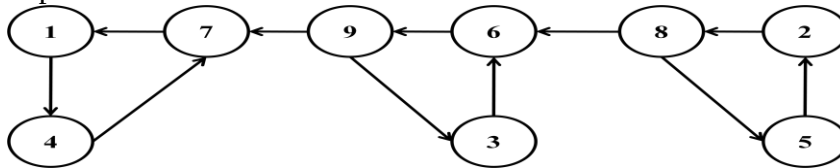
- In social networks, a group of people are generally strongly connected (For example, students of a class or any other common place). Many people in these groups generally like some common pages or play common games. The SCC algorithms can be used to find such groups and suggest the commonly liked pages or games to the people in the group.

○ **University Questions**

1. Define Strongly Connected Components of a graph. Give one example.
2. Define Strongly Connected Components of a graph. Write the algorithm to find Strongly Connected Components in a graph
3. Define Strongly Connected Components of a graph. How DFS can be used to find strongly connected components
4. Find strongly connected components of the digraph given below:



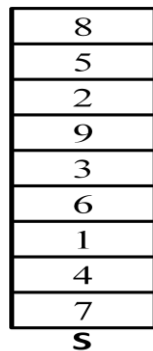
5. Find strongly connected components of the digraph using the algorithm showing each step



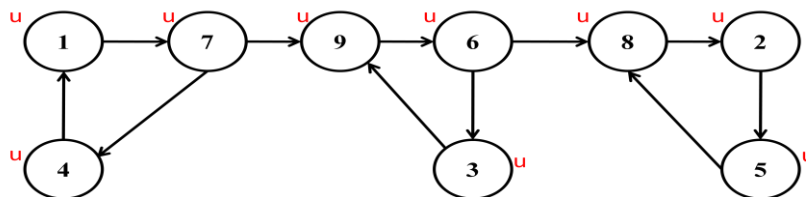
**Answer:**

Pass 1

DFS: 1,4,7,9,3,6,8,5,2



Reverse the graph



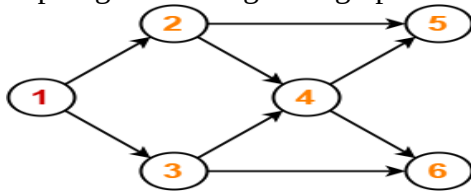
Connected Components: 8-2-5

9-6-3

1-7-4

- **Topological Sorting**

- Topological sorting for Directed Acyclic Graph (DAG) is a linear ordering of vertices such that for every directed edge (u,v), vertex u comes before v in the ordering.
- A topological sort of a graph is an ordering of its vertices along a horizontal line so that all directed edges go from left to right.
- If the graph contains a cycle, then no linear ordering is possible.
- Topological Sorting for a graph is not possible if the graph is not a DAG.



**Topological Sort Example**

One possible Topological Sort=[1,2,3,4,5,6]

- **Algorithm**

1. Identify a node with no incoming edges(indegree=0)
2. Add this node to the ordering.
3. Remove this node and all its outgoing edges from the graph.
4. Repeat step 1 to 3 until the graph becomes empty

- **Complexity**

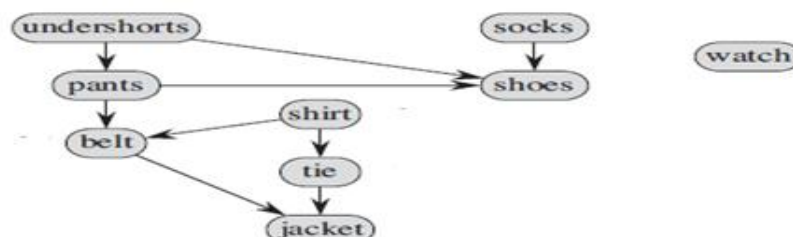
- Suppose  $|E|$  is the number of edges and  $|V|$  is the number of nodes of the graph G.
- Time to determine the indegree for each node =  $O(E)$  time. This involves looking at each directed edge in the graph once.
- Time to determine the nodes with no incoming edges =  $O(V)$  time
- Add nodes until we run out of nodes with no incoming edges. This loop could run once for every node— $O(V)$  times
  - Constant-time operations to add a node to the topological ordering.
  - Decrement the indegree for each neighbor of the node we added. Over the entire algorithm, we'll end up doing exactly one decrement for each edge, making this step  $O(E)$  time.
- Check if we included all nodes or found a cycle. This is a fast  $O(1)$  comparison
- All together, the time complexity is  $O(V+E)$

- **Applications**

- Scheduling jobs from the given dependencies among jobs
- Instruction Scheduling
- Determining the order of compilation tasks to perform in makefiles
- Data Serialization

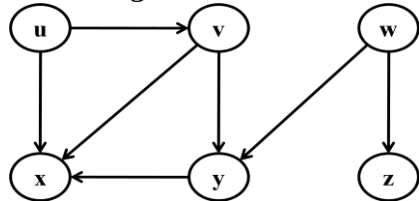
- **Examples**

1. Write the topological sorting for the DAG given below



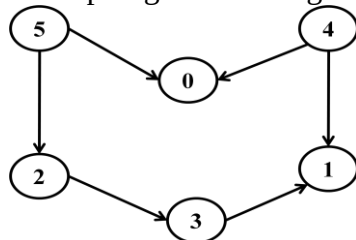
○ **University Questions**

1. What is meant by topological sorting? Write the algorithm to do topological sorting in a directed acyclic graph
2. Consider the directed acyclic graph  $G=(V,E)$  given in the following figure. Find any topological ordering of  $G$



**Answer:** u, v, w, z, y, x

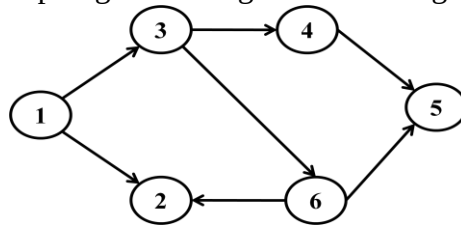
3. Find any two topological ordering of the DAG given below



**Answer:** 5, 2, 3, 4, 1, 0

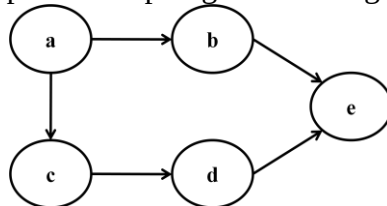
5, 2, 3, 4, 0, 1

2. Write the topological sorting for the DAG given below



**Answer:** 1, 3, 4, 6, 2, 5

3. Find the possible topological orderings for the following graph

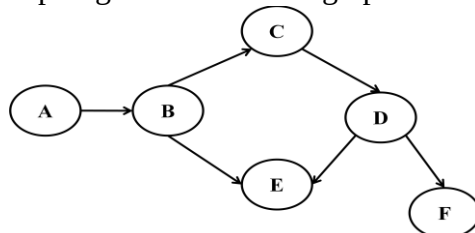


**Answer:** a, b, c, d, e

a, c, b, d, e

a, c, d, b, e

4. Write the topological order of the graph  $G$



**Answer:** A, B, C, D, E, F

- **Disjoint Sets**

- Two or more sets with nothing in common are called disjoint sets.
- Example:  $S1 = \{1, 2, 3, 4\}$   $S2 = \{5, 6, 7\}$   $S3 = \{8, 9\}$
- Two sets  $S1$  and  $S2$  are said to be disjoint if  $S1 \cap S2 = \phi$
- The disjoint set data structure is also known as union-find data structure and merge-find set.

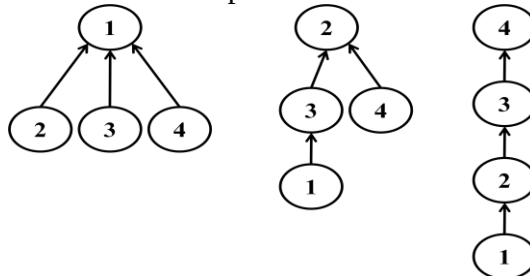
- **Disjoint set representations**

- Linked list representation(Tree representation)
- Array representation

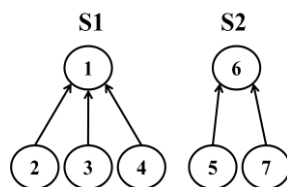
- **Linked list representation(Tree representation)**

- Example:  $S1 = \{1, 2, 3, 4\}$

This set can be represented as tree in different ways



- Example:  $S1 = \{1, 2, 3, 4\}$   $S2 = \{5, 6, 7\}$



- **Array representation**

- Example:  $S1 = \{1, 2, 3, 4\}$   $S2 = \{5, 6, 7\}$

|   |    |   |   |   |   |   |    |   |
|---|----|---|---|---|---|---|----|---|
| i | 1  | 2 | 3 | 4 | 5 | 6 | 7  |   |
| p | -1 | 1 | 1 | 1 | 1 | 6 | -1 | 6 |

- **Disjoint Set operations**

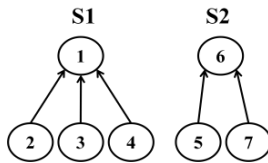
- Make set
- Union
- Find

- **Make Set Operation**

- Make-set(x) function will create a set that containing one element x.
- **Algorithm Make-set(x)**
  1. Create a new linked list that contains one node n
  2.  $n \rightarrow \text{data} = x$
  3.  $n \rightarrow \text{parent} = \text{NULL}$

- **Find Operation**

- Determine which subset a particular element is in.
- This will return the representative(root) of the set that the element belongs.
- This can be used for determining if two elements are in the same subset.



Find(3) will return 1, which is the root of the tree that 3 belongs

Find(6) will return 6, which is the root of the tree that 6 belongs

- **Find Algorithm**

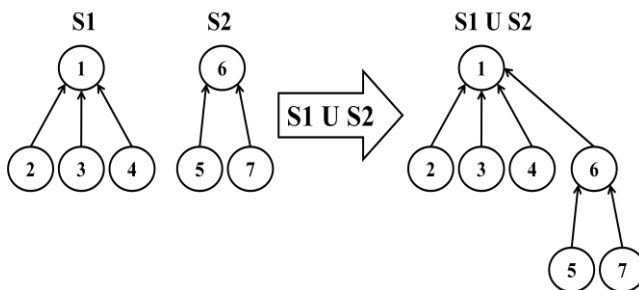
Algorithm Find(n)

1. while  $n \rightarrow \text{parent} \neq \text{NULL}$  do
  - 1.1  $n = n \rightarrow \text{parent}$
2. return n

**Worst case Time Complexity =  $O(d)$** , where d is the depth of the tree

- **Union Operation**

- Join two subsets into a single subset.
- Here first we have to check if the two subsets belong to same set. If no, then we cannot perform union



| i | 1  | 2 | 3 | 4 | 5 | 6 | 7 |
|---|----|---|---|---|---|---|---|
| P | -1 | 1 | 1 | 1 | 6 | 1 | 6 |

- **Union Algorithm**

Algorithm Union(a, b)

1.  $X = \text{Find}(a)$
2.  $Y = \text{Find}(b)$
3. If  $X \neq Y$  then
  1.  $Y \rightarrow \text{parent} = X$

**Worst case Time Complexity =  $O(d)$** , where d is the depth of the tree

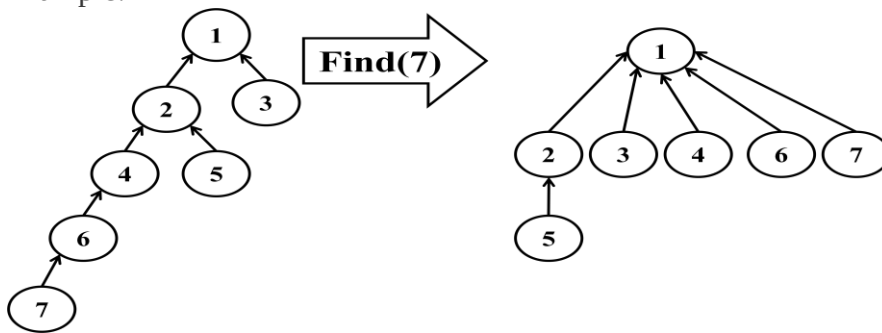
- **There are two ways to improve the time complexity of Find and Union operation:**

1. Path Compression
2. Union by Rank

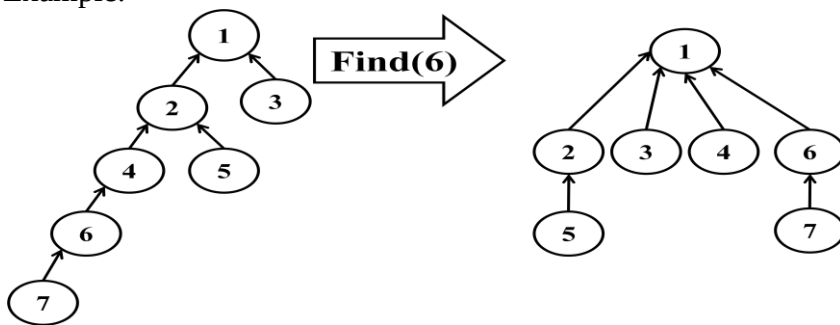
- **Path Compression(collapsing rule)**

- Path compression is a way of flattening the structure of the tree whenever Find is used on it.

- Since each element visited on the way to a root is part of the same set, all of these visited elements can be reattached directly to the root.
- Example:

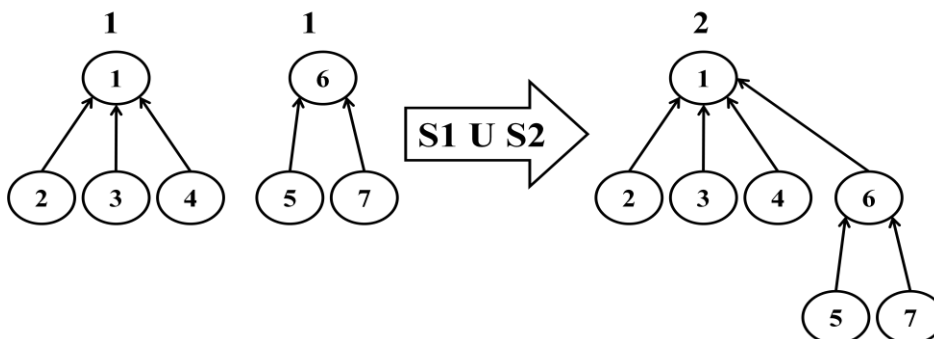


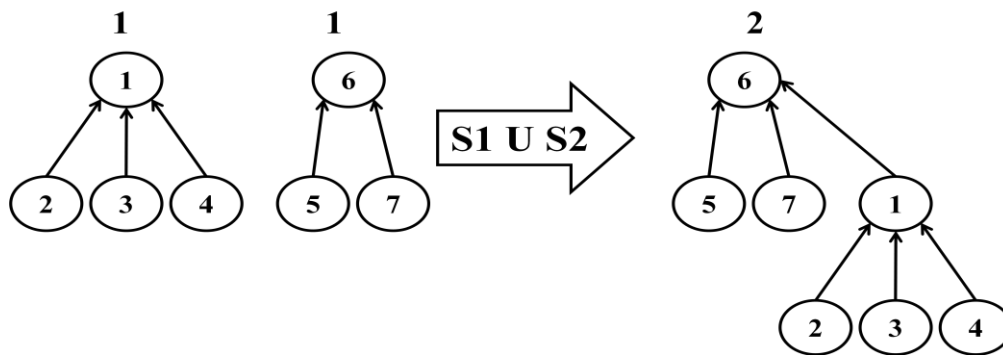
- Next time we perform Find(6) it will give us the answer in two steps instead of four prior to the optimisation.
- Example:



#### ○ Union by Rank(weighted rule)

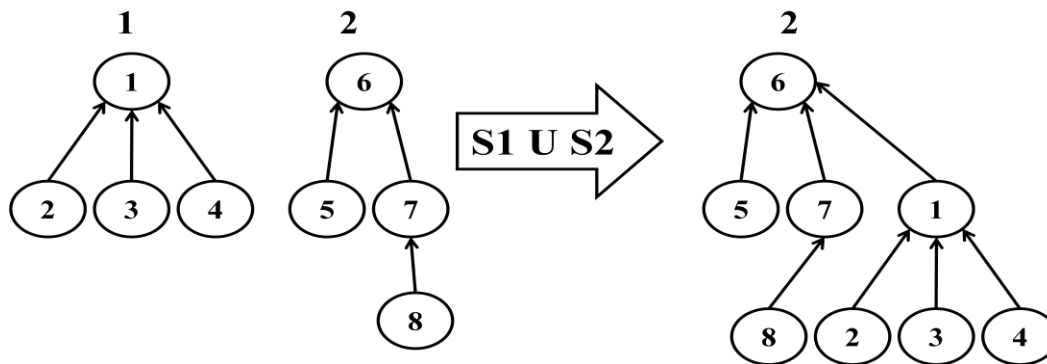
- In this operation we decide which tree gets attached to which.
- The basic idea is to keep the depth of the tree as small as possible.
- We assign a new value **Rank** to each set. This rank represents the depth of the tree that the given set represents.
- If we union two sets and :
  - Both sets have same rank → then the resulting set's rank is 1 larger.
  - Both sets have the different ranks — the resulting set's rank is the larger of the two.
- Example:





We can Union these two sets in the above two ways.

- Example:



#### ○ Applications of disjoint sets

- It is easier to check whether the given two elements are belongs to the same set.
- Used to merge 2 sets into one

#### ○ University Questions

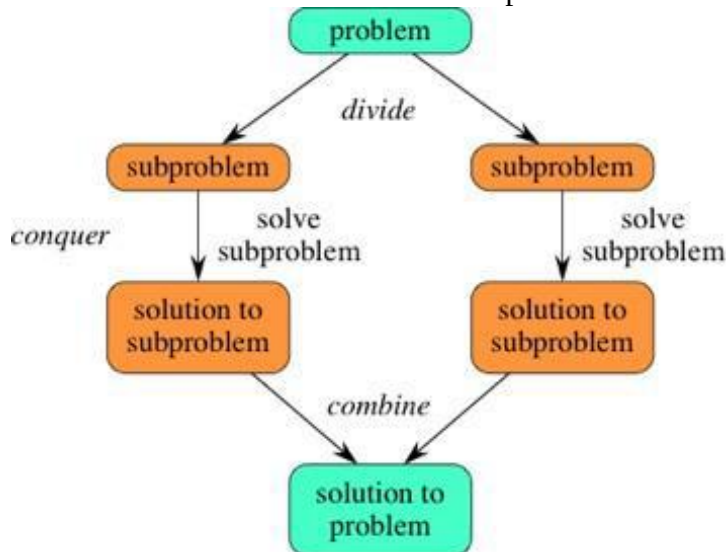
1. Implement UNION using linked list representation of disjoint sets.
2. Show the UNION operation using linked list representation of disjoint sets
3. Explain the UNION and FIND-SET operations in the linked-list representation of disjoint sets. Discuss the complexity
4. State weighted rule (union by rank) and collapsing rule (path compression) applied in the disjoint set union and find operation respectively. How these rules will improve the efficiency of disjoint set operations
5. Discuss briefly the heuristics, union by rank and path compression, to improve the running time of disjoint set data structure

- **Divide & Conquer and Greedy Strategy**

- **The Control Abstraction of Divide and Conquer**
- **2-way Merge sort**
- **Strassen's Algorithm for Matrix Multiplication-Analysis**
- **The Control Abstraction of Greedy Strategy**
- **Fractional Knapsack Problem**
- **Minimum Cost Spanning Tree Computation- Kruskal's Algorithms – Analysis**
- **Single Source Shortest Path Algorithm - Dijkstra's Algorithm-Analysis**

- **Divide and Conquer**

- Divide and conquer algorithm is having three parts:
  1. **Divide** the problem into a number of sub-problems that are smaller instances of the same problem.
  2. **Conquer** the sub-problems by solving them recursively. If they are small enough, solve the sub-problems as base cases.
  3. **Combine** the solutions to the sub-problems into the solution for the original problem.



- Control Abstraction: It is a procedure whose flow of control is clear but whose primary operations are specified by other procedure whose precise meanings are left undefined.
- **Control Abstraction: Divide and Conquer**

```

Algorithm DAndC(P) {
    if Small(P) then
        return S(P)

    else
    {
        Divide P into smaller instances  $P_1, P_2, \dots, P_k$ ,  $k \geq 1$ ; apply
        DAndC to each of these sub-problems;
        return Combine(DAndC( $P_1$ ), DAndC( $P_2$ ),  $\dots$ , DAndC( $P_k$ ));
    }
}
  
```

- If the given problem is small, return the result

- Otherwise, divide the problem into smaller instances  $P_1, P_2, \dots, P_k$
- Apply DAndC() to each of these sub-problems.
- Finally combine the results of all sub-problems.

- DAndC() can be described using the following recurrence relation:

$$T(n) = \begin{cases} g(n) & n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{Otherwise} \end{cases}$$

- $T(n)$ : Time for divide and conquer on any input of size  $n$
- $f(n)$ : Complexity of dividing the problem and combining the results.
- Complexity of many divide and conquer algorithms are given by the following recurrence relation

$$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

## ○ 2 Way Merge Sort

- Given a sequence of  $n$  elements  $a[1], \dots, a[n]$ . Split this array into two sets  $a[1], \dots, a[n/2]$  and  $a[(n/2)+1], \dots, a[n]$ . Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of  $n$  element.

```

Algorithm MergeSort(low, high)
{
    mid = (low + high)/2;
    MergeSort(low, mid);
    MergeSort(mid+1, high);
    Merge(low, mid, high);
}

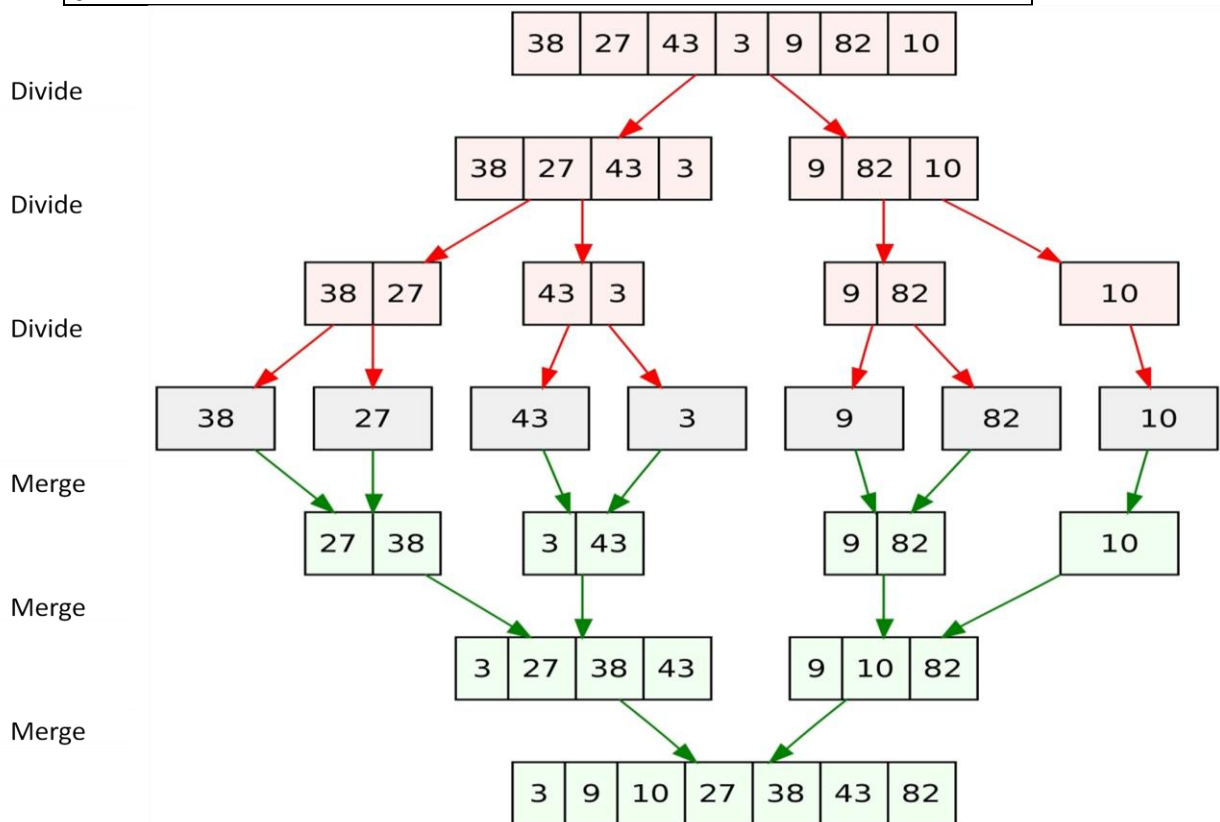
Algorithm Merge(low, mid, high)
{
    i= low; x= low; y= mid + 1;
    while((x ≤ mid) and (y ≤ high)) do
    {
        if ( a[x] ≤ a[y] ) then
        {
            b[i] = a[x];
            x = x+1;
        }
        else
        {
            b[i] = a[y];
            y = y+1;
        }
        i=i+1;
    }
    if( x ≤ mid) then
    {

```

```

    for k=x to mid do
    {
        b[i] = a[k];
        i =i+1;
    }
    }
    else
    {
        for k=y to high do
        {
            b[i] = a[k];
            i =i+1;
        }
    }
    for k= low to high do
        a[k] = b[k];
}

```



▪ **Complexity**

$$T(n) = \begin{cases} a & \text{if } n=1 \\ 2 T(n/2) + cn & \text{Otherwise} \end{cases}$$
 a is the time to sort an array of size 1  
 cn is the time to merge two sub-arrays  
 2 T(n/2) is the complexity of two recursion calls

$$\begin{aligned}
 T(n) &= 2 T(n/2) + c n \\
 &= 2(2 T(n/4) + c(n/2)) + c n
 \end{aligned}$$

$$\begin{aligned}
&= 2^2 T(n/2^2) + 2 c n \\
&= 2^3 T(n/2^3) + 3 c n \\
&\dots\dots\dots = 2^k T(n/2^k) + k c n \quad \text{[Assume that } 2^k = n \rightarrow \\
&\quad k = \log n] \\
&= n T(1) + c n \log n \\
&= a n + c n \log n \\
&= \mathbf{O(n \log n)} \text{ Best Case, Average Case and Worst Case Complexity of Merge Sort = } \\
&\mathbf{O(n \log n)}
\end{aligned}$$

○ **Divide and Conquer Matrix Multiplication**

- Native matrix multiplication complexity =  $O(n^3)$

- **Divide and Conquer Matrix Multiplication Algorithm**

1. We have to compute the product of 2 nxn matrices A and B.
2. Assume that n is the power of 2. That is  $n=2^k$   
If n is not a power of 2, then enough rows and columns of 0's can be added to both A and B so that the resulting dimensions are the power of two.
3. Then partition A and B into 4 square matrices, each of size  $n/2 \times n/2$
4. AB can be computed using the formula

$$C_{11} = A_{11} B_{11} + A_{12} B_{21}$$

$$C_{12} = A_{11} B_{12} + A_{12} B_{22}$$

$$C_{21} = A_{21} B_{11} + A_{22} B_{21}$$

$$C_{22} = A_{21} B_{12} + A_{22} B_{22}$$

5. If  $n=2$ , these formulas are computed using a multiplication operation for the elements of A and B
6. If  $n>2$ , the elements of C can be computed using matrix multiplications and addition operations applied to the matrices of size  $n/2 \times n/2$
7. This algorithm will continue applying itself to smaller sized sub-matrices until n becomes suitably small( $n=2$ ) so that the product is computed directly.

- **Complexity**

- For multiplying two matrices of size  $n \times n$ , we make 8 recursive calls above, each on a matrix with size  $n/2 \times n/2$ .
- Addition of two matrices takes  $O(n^2)$  time.
- Time complexity =  $8 T(n/2) + O(n^2)$   
=  $\mathbf{O(n^3)}$  [By Master's Theorem]

○ **Strassen's Matrix Multiplication**

- **Algorithm**

1. A and B are the matrices with dimension  $n \times n$
2. If n is not a power of 2, then enough rows and columns of 0's can be added to both A and B so that the resulting dimensions are the power of two.
3. Partition A and B into 4 square matrices of size  $n/2 \times n/2$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

A                      B

a, b, c and d are sub-matrices of A, of size  $n/2 \times n/2$

e, f, g and h are sub-matrices of B, of size  $n/2 \times n/2$

4. Compute 7  $n/2 \times n/2$  matrices

$$P_1 = a (f - h)$$

$$P_2 = h (a + b)$$

$$P_3 = e (c + d)$$

$$P_4 = d (g - e)$$

$$P_5 = (a + d) (e + h)$$

$$P_6 = (b - d) (g + h)$$

$$P_7 = (a - c) (e + f)$$

It requires 7 matrix multiplications and 10 matrix additions and subtractions

5. Then compute C

$$C = \begin{pmatrix} C_1 & C_2 \\ C_3 & C_4 \end{pmatrix}$$

$$C_1 = P_4 + P_5 + P_6 - P_2$$

$$C_2 = P_1 + P_2$$

$$C_3 = P_3 + P_4$$

$$C_4 = P_1 - P_3 + P_5 - P_7$$

#### ▪ Complexity

- For multiplying two matrices of size  $n \times n$ , we make 7 matrix multiplications and 10 matrix additions and subtractions
- Addition/Subtraction of two matrices takes  $O(n^2)$  time.
- Time complexity =  $7 T(n/2) + O(n^2)$   
 $= O(n^{\log_2 7}) = O(n^{2.81})$  [By Master's Theorem]

$$T(n) = \begin{cases} b & \text{if } n < 2 \\ 7 T(n/2) + c n^2 & \text{Otherwise} \end{cases}$$

$$\begin{aligned} T(n) &= 7 T(n/2) + c n^2 \\ &= 7^2 T(n/2^2) + 7 c n^2/4 + c n^2 \\ &= 7^3 T(n/2^3) + 7^2 c n^2/4^2 + 7 c n^2/4 + c n^2 \\ &\dots\dots\dots \\ &= 7^k T(n/2^k) + (7^{k-1}/4^{k-1}) c n^2 + \dots\dots\dots + (7/4) c n^2 + c n^2 \\ &= 7^k T(n/2^k) + [1 + (7/4) + \dots\dots\dots + (7^{k-1}/4^{k-1})] c n^2 \\ &\leq 7^k T(n/2^k) + [1 + (7/4) + \dots\dots\dots] c n^2 \\ &= 7^k T(n/2^k) + [1/(1-(7/4))] c n^2 \\ &= 7^{\log_2 n} T(1) - [4/3] c n^2 \quad \text{[Assume that } n/2^k = 1 \rightarrow k = \log n] \\ &= n^{\log_2 7} O(1) - [4/3] c n^2 \\ &= O(n^{\log_2 7}) = O(n^{2.81}) \end{aligned}$$